

NTI · MEAN STACK · GRADUATION PROJECT

Dawwarha — Backend Implementation Plan

A practical execution guide for the Node.js, Express, MongoDB, and Mongoose backend.

Version 1.0 · Prepared for the development team, QA, and product review
Generated September 9, 2026

Contents

Click any section to jump to it in the PDF.

01 — Executive Summary

02 — Architecture

03 — Technology Stack

04 — Project Structure

05 — Domain Map

06 — Database Architecture

07 — Authentication & Authorization

07.1 Authentication (FR-001–FR-003, NFR-002, NFR-003)

07.2 Authorization / RBAC

08 — Engineer 1 Plan — رحمة محمد محمد أحمد

TASK 1.A — Authentication Middleware & Service

TASK 1.B — RBAC / `authorize` Middleware

09 — Engineer 2 Plan — ادهم محمد حسن عبد السميع

TASK 2.A — Resource CRUD Routes + Ownership Enforcement

10 — Engineer 3 Plan — دينا عبدالله عليان محمد

TASK 3.A — Request CRUD Routes

TASK 3.B — Matching Engine HTTP Surface

11 — Engineer 4 Plan — عبدالستار محمد عبدالستار مطاوع

TASK 4.A — Handover Confirmation Endpoint

12 — Cross-Engineer Contracts

13 — Transactions

14 — API Specification

15 — Validation Strategy

16 — Error Handling

17 — Security

18 — Testing Strategy

19 — Seed Data

20 — Git Workflow

21 — Integration Plan

22 — Deployment Preparation

23 — Documentation

24 — Definition of Done

25 — Complete Task Checklist

26 — Critical Path

27 — Risks & Common Failure Points

28 — Final Graduation Demo Flow

Graduation Project · NTI · MEAN Stack Version 1.0 — Derived from the Product Brief v1.0 and the MongoDB Implementation Plan v1.0 Audience: 4 beginner-level backend engineers, executing from an empty backend repository to a graduation-ready demo

Team:

Engineer	Name	Domain
1	رحمة محمد محمد أحمد	Identity & Organizations
2	ادهم محمد حسن عبد السميع	Supply (Resources & Categories)
3	دينا عبدالله عليان محمد	Demand & Matching (Requests & Matches)
4	عبدالستار محمد عبدالستار مطاوع	Transfer, Trust & Impact

01 — Executive Summary

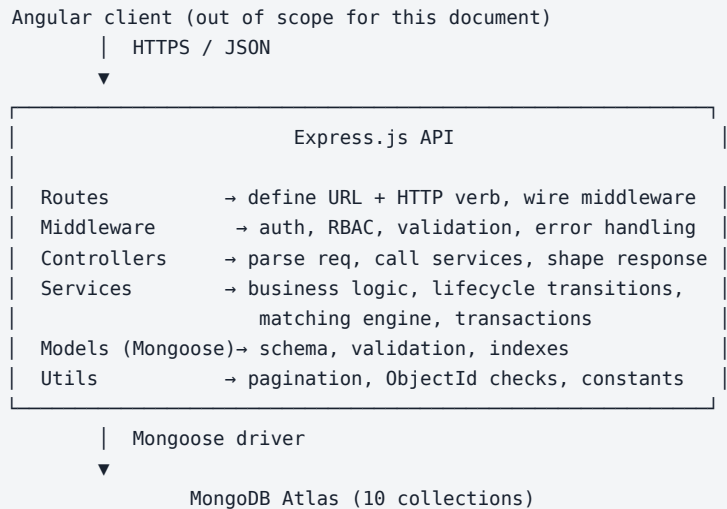
Dawwarha's backend is a Node.js + Express + MongoDB (Mongoose) REST API that implements the product's core promise: a resource offered by one party can be discovered, scored against open requests, matched, and safely handed over — with a two-sided confirmation and an auditable impact record at the end.

This document is the **execution guide** for the backend half of the graduation project. It takes the database design already produced ([Dawwarha – MongoDB Implementation Plan v1.0](#)) as its source of truth for schemas, indexes, and embedding/referencing decisions, and **does not re-litigate those decisions** — it builds the missing layers on top of them: routes, controllers, services, middleware, authentication, authorization/RBAC, state-machine enforcement, the matching engine's HTTP surface, transactions, error handling, security, testing, seed data, git workflow, API documentation, and the final integration/demo plan.

The plan preserves the four-way domain ownership already established in the database plan (Identity, Supply, Demand & Matching, Transfer/Trust/Impact) and extends it consistently into the API and service layers, so that each engineer owns their collections **and** the routes/controllers/services that operate on them, end to end.

Guiding principle throughout: **do not over-engineer the MVP**. No microservices, no message queues, no Redis, no GraphQL, no ML — a well-structured monolithic Express API with MongoDB transactions used only where multi-document consistency genuinely requires them is sufficient, and is what the Product Brief and the DB plan both explicitly call for.

02 — Architecture



Request lifecycle for every write endpoint:

```

Request
↓
Route (matches method + path)
↓
Middleware: authenticate (JWT) → authorize (role/ownership) → validate (schema)
↓
Controller (thin: parse req, call service, send response)
↓
Service (business logic: lifecycle rules, scoring, transactions)
↓
Model / MongoDB (persist, enforce schema-level validation)
↓
Response ( { success, data } or { success: false, error } )
  
```

Separation of concerns

Layer	Responsibility	Must NOT contain
Routes	URL → middleware chain → controller mapping only	business logic, direct DB calls
Middleware	Cross-cutting concerns: auth, RBAC, validation, error formatting	domain-specific business rules
Controllers	Read req, call exactly one service function, shape the HTTP response	Mongoose queries, lifecycle logic

Layer	Responsibility	Must NOT contain
Services	All business logic: lifecycle transitions, matching, transactions, cross-collection coordination	<code>req / res</code> objects (services never see Express objects)
Models	Mongoose schemas, field-level validation, indexes	business rules that depend on more than one document
Validators	Request-payload shape validation (Joi/express-validator or manual)	persistence logic
Utilities	Pagination helpers, ObjectId checks, constants (weights, thresholds)	anything stateful

This mirrors the Product Brief's own architecture section (backend: routes/controllers/services/middleware) and the DB plan's file structure (Section 19 of that document) — this plan is additive, not a rewrite.

03 — Technology Stack

Concern	Choice	Why
Runtime	Node.js (LTS)	Required by the MEAN stack / NTI brief
Framework	Express.js	Required; minimal, well understood by beginners
Database	MongoDB Atlas	Already the DB plan's foundation
ODM	Mongoose	Schema validation, <code>.populate()</code> , transactions via sessions
Auth	JWT (<code>jsonwebtoken</code>) + <code>bcrypt</code>	Stateless sessions, industry-standard password hashing
Validation	Mongoose schema validation (persistence layer) + a lightweight request validator (<code>express-validator</code> or hand-written validators — team's choice, but one choice, not mixed)	Two enforcement points per Product Brief Section 21

Concern	Choice	Why
Testing	Jest or Mocha+Chai (team picks one, used consistently) + <code>supertest</code> for API tests	Beginner-friendly, widely documented
Docs	Postman collection (exported JSON, committed to repo) or a lightweight OpenAPI/Swagger file if time allows	"Prefer OpenAPI/Swagger if appropriate and lightweight"
Env config	<code>dotenv</code>	<code>.env</code> files, never committed
Logging	Built-in <code>console.*</code> wrapped by a tiny logger util, or <code>morgan</code> for HTTP request logs	Lightweight, no external log aggregation needed for a graduation demo

Explicitly **not used**, per both source documents: microservices, Kafka/RabbitMQ, Redis, Kubernetes, GraphQL, ML/embedding-based matching, vector databases, external recommendation APIs, unnecessary cloud infra.

If the repository already pins a specific validation or testing library before this plan is executed, **follow the repository** — this table is the default when no prior choice exists.

04 — Project Structure

```

backend/
├── src/
│   ├── config/
│   │   └── database.js          # Mongo connection (Eng 1, Task 1.1)
│   ├── models/
│   │   ├── User.js             # Eng 1
│   │   ├── Organization.js     # Eng 1
│   │   ├── Category.js        # Eng 2
│   │   ├── Resource.js        # Eng 2
│   │   ├── Request.js         # Eng 3
│   │   ├── Match.js           # Eng 3
│   │   ├── Handover.js        # Eng 4
│   │   ├── Report.js          # Eng 4
│   │   ├── Notification.js     # Eng 4
│   │   └── Contribution.js     # Eng 4
│   ├── routes/
│   │   ├── auth.routes.js      # Eng 1
│   │   ├── users.routes.js     # Eng 1
│   │   ├── organizations.routes.js # Eng 1
│   │   ├── categories.routes.js # Eng 2
│   │   ├── resources.routes.js  # Eng 2
│   │   ├── requests.routes.js   # Eng 3
│   │   ├── matches.routes.js    # Eng 3
│   │   ├── transactions.routes.js # Eng 4 (handover confirmation)
│   │   ├── reports.routes.js    # Eng 4
│   │   ├── notifications.routes.js # Eng 4
│   │   └── admin.routes.js      # shared – mounts admin-only sub-resources
│   ├── controllers/            # one file per routes file, same owner
│   ├── services/
│   │   ├── authService.js       # Eng 1
│   │   ├── organizationService.js # Eng 1
│   │   ├── resourceLifecycleService.js # Eng 2 (from DB plan)
│   │   ├── requestLifecycleService.js # Eng 3 (from DB plan)
│   │   ├── matchingService.js   # Eng 3 (from DB plan)
│   │   ├── handoverService.js   # Eng 4 (from DB plan)
│   │   ├── reportService.js     # Eng 4
│   │   ├── notificationService.js # Eng 4
│   │   └── contributionService.js # Eng 4 (from DB plan)
│   ├── middleware/
│   │   ├── authenticate.js      # verifies JWT, attaches req.user – Eng 1
│   │   ├── authorize.js         # role + ownership checks – Eng 1, consumed by all
│   │   ├── validate.js         # generic "run this schema" wrapper – Eng 1, consumed by all
│   │   └── errorHandler.js     # centralized error middleware – Eng 1
│   └── validators/             # per-endpoint payload validators, one file per domain, owned like
models
├── utils/
│   ├── pagination.js
│   ├── objectId.js
│   ├── constants.js           # matching weights/thresholds, pagination defaults
│   └── apiResponse.js         # success()/error() response shape helpers
├── seed/
│   └── seed.js
├── app.js                     # Express app assembly (middleware order, route mounting)
├── server.js                  # entry point: connectDB() then app.listen()
├── tests/
│   ├── unit/
│   └── integration/
├── .env.example

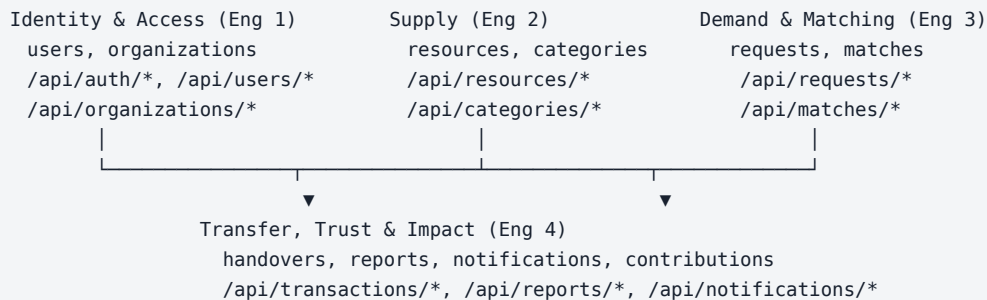
```

```
├─ .gitignore
├─ package.json
```

This is a direct extension of the DB plan's Section 19 file structure with `routes/`, `controllers/`, `middleware/`, and `validators/` added — the model ownership map is unchanged, and route/controller/service ownership follows the same collection owner.

05 — Domain Map

Identical to the DB plan's domain map (Section 04 of that document), extended with the API surface each domain now also owns:



`categories` stays under Engineer 2 (Supply), read-only for Engineer 3, exactly as decided in the DB plan — restated here rather than re-argued.

06 — Database Architecture

The full field-by-field schema, index strategy, embedding/referencing rationale, aggregation strategy, and data-integrity rules are already specified in **Dawwarha – MongoDB Implementation Plan v1.0** (Sections 06–16 of that document) and are adopted here **without modification**. Every engineer must read the sections matching their domain before writing any model code:

Collection	Owner	DB Plan Section
users, organizations	Engineer 1	6.1, 6.2, 8.1, 8.2
categories, resources	Engineer 2	6.3, 6.4, 8.3, 8.4

Collection	Owner	DB Plan Section
requests, matches	Engineer 3	6.5, 6.6, 8.5, 8.6
handovers, reports, notifications, contributions	Engineer 4	6.7–6.10, 8.7–8.10

Summary only (see the DB plan for the authoritative detail):

- **10 collections**, each with exactly one owner, relationships expressed via `ObjectId` references except for a small number of justified embeds (`resource.location`, `resource.availabilityWindow`, `match.scoreBreakdown`, `organization.verification`, `users.stats` cache) — see DB plan Section 07 for the six-question justification behind each.
- **24 indexes total**, every one mapped to a named query (DB plan Section 10) — no speculative indexes are to be added by any engineer without updating that document first.
- **Six lifecycle state machines** (`resources.status`, `requests.status`, `matches.status`, `handovers.status`, `organizations.verification.status`, `reports.status`) — enforced centrally by table-driven service functions, never by scattered `if` statements in controllers (DB plan Section 09; restated for the API layer in Section 12 below).
- **One deliberate denormalization**: `matches.providerId` / `matches.requesterId` copied from `resources.providerId` / `requests.requesterId` for query performance, safe because those source fields are immutable after creation (DB plan Section 07.4).
- **One MongoDB transaction boundary that matters**: match acceptance (`matches` + `resources` + `handovers` in one transaction) — see Section 13 below.

Any backend task that appears to require a new collection, field, enum value, or index not listed in the DB plan must **update that document first** via the Shared Database Contract process (DB plan Section 18), not improvise silently.

07 — Authentication & Authorization

07.1 Authentication (FR-001–FR-003, NFR-002, NFR-003)

- **Registration** (`POST /api/auth/register`): validate payload → check `email` uniqueness → hash password with `bcrypt` (cost factor 10–12) → create `User` with `role: 'user'`, `status: 'active'` → return a JWT. Never accept a `role` or `status` field from the request body — always set server-side.
- **Login** (`POST /api/auth/login`): find user by email (must explicitly `.select('+passwordHash')` since the schema marks it `select: false`) → `bcrypt.compare`

→ if `status === 'suspended'`, reject with 403 regardless of correct password → sign and return a JWT (`sub: user._id`, `role`, short-to-medium expiry, e.g. 7 days for an MVP demo).

- **Logout:** stateless JWT — "logout" is a client-side token discard. Documented as such; no server-side token blacklist is built for the MVP (would need Redis or a DB collection — out of scope, not required by any FR).
- **Password hashing:** `bcrypt` only, hashing happens in `authService`, never in a Mongoose pre-save hook that could accidentally re-hash an already-hashed password on unrelated profile updates (see DB plan Task 1.2, mistake #3).
- **Token expiration:** on expiry, `authenticate` middleware returns 401 with a message the frontend maps to "Please log in to continue" (Product Brief Section 22).
- **Email verification:** schema supports `emailVerifiedAt` (DB plan 6.1); MVP can ship without blocking login on it (Product Brief lists it as basic, not gating) — document whichever the team decides, but do not silently drop the field.

07.2 Authorization / RBAC

Three roles, exactly as the Product Brief defines them (Section 14): `user`, `organization` (not a `users` role — it is verification status on the `organizations` collection, tied to a user via `ownerUserId`, per DB plan Section 6.2), and `admin`.

authorize middleware has two independent checks, always composed explicitly per route — never assumed:

1. **Role check** — `authorize.requireRole('admin')` — is this account type allowed to hit this endpoint at all?
2. **Ownership check** — `authorize.requireOwnership(getOwnerId)` — for a specific document, does `req.user._id` match the document's `providerId` / `requesterId` / `ownerUserId` / `reporterId`? Admins bypass ownership checks; everyone else must pass them.

Suspended users: `authenticate` itself rejects any request from a `status: 'suspended'` user with 403, **before** role/ownership checks run — a suspended user should never reach a controller.

Organization-gated actions (publishing a standing request as an org) additionally check `organization.verification.status === 'approved'` — this is a business rule, not a role check, so it lives in `organizationService` / `requestLifecycleService`, not in the generic `authorize` middleware.

Permission matrix (built from Product Brief Section 14 + DB plan CRUD ownership fields):

Capability	User	Organization	Admin
Register / Login	✓	✓ (via owner user)	✓

Capability	User	Organization	Admin
Create Resource	✓	✓ (as <code>providerOrgId</code>)	—
Update/Delete own Resource	✓ (owner)	✓ (owner)	✓ (any)
Create Request	✓	✓ (as <code>requesterOrgId</code>)	—
Publish standing/recurring Request	—	✓ (if verified)	—
Generate / Accept / Reject Match	✓ (party to it)	✓ (party to it)	—
Confirm Handover	✓ (provider or seeker on it)	✓	—
Create Report	✓	✓	✓
Resolve Report	—	—	✓
Register Organization	✓ (becomes owner)	—	—
Approve/Reject Organization Verification	—	—	✓
Manage Categories (CRUD)	—	—	✓
Suspend/Reactivate User	—	—	✓
View own Contribution History	✓	✓	✓
View Platform Analytics	—	limited (own org)	✓ (full)

Every "✓ (owner)" or "party to it" cell is enforced by the `authorize.requireOwnership` middleware reading the specific field named in the DB plan (`providerId` , `requesterId` , `reporterId` , `ownerUserId` , `providerId / seekerId` on `handovers`) — never by trusting the client.

08 — Engineer 1 Plan — رحمة محمد محمد أحمد

Mission: Identity & Organizations. Owns `users` , `organizations` , and the platform-wide auth/RBAC plumbing every other engineer's routes depend on (`authenticate` , `authorize` ,

`errorHandler` , `database.js`).

Prerequisite reading: DB plan Sections 6.1, 6.2, 8.1, 8.2, 9.1, 9.6 (already-built models — this plan adds the API layer on top; do not re-derive the schemas).

TASK 1.A — Authentication Middleware & Service

Owner: Engineer 1

Goal:

Build the authenticate middleware and authService that every protected route in the whole application depends on.

Why this exists:

Without a single, shared way to verify "who is making this request," every other engineer would have to reinvent token checking – inconsistently and insecurely.

Prerequisites:

User model complete (DB plan Task 1.2). JWT_SECRET present in .env.

Files to create:

```
src/services/authService.js
src/middleware/authenticate.js
src/controllers/auth.controller.js
src/routes/auth.routes.js
src/validators/auth.validators.js
```

Files to modify:

```
src/app.js (mount /api/auth routes)
```

Step 1 – authService.register(payload):

Validate email/password shape (delegate to auth.validators.js), check for an existing user with that email, hash the password with bcrypt (cost 10), create the User document with role: 'user' hardcoded (never taken from payload), return { user, token }.

Step 2 – authService.login(email, password):

Find user with .select('+passwordHash'). If not found OR bcrypt.compare fails, throw the SAME generic "invalid credentials" error either way – do not reveal whether the email exists (a security best practice beginners often miss). If user.status === 'suspended', throw a distinct 403 "account suspended" error. On success, sign a JWT with { sub: user._id, role: user.role } and return { user (without passwordHash), token }.

Step 3 – authService.signToken(user) / verifyToken(token):

Small wrappers around jsonwebtoken.sign / .verify, expiry read from an env var (JWT_EXPIRES_IN, default '7d').

Step 4 – authenticate middleware:

Read Authorization: Bearer <token> header. Missing header → 401. Invalid/expired token → 401. Valid token → look up the user by the token's sub, attach as req.user (excluding passwordHash). If the freshly-loaded user is now suspended, reject with 403 even if the token itself is still valid (a token issued before suspension must not keep working).

Step 5 – Routes:

POST /api/auth/register → controller calls authService.register, returns 201 + { user, token }.
 POST /api/auth/login → controller calls authService.login, returns 200 + { user, token }.
 POST /api/auth/logout → returns 200 with a message; stateless, no server action required (documented explicitly as such in the response/comment so nobody "fixes" this later by adding fake statefulness).

Business rules:

role and status are never accepted from the client on register.
 Duplicate email → 409 Conflict, not 500.

Validation rules:

email: required, valid email format. password: required, minimum 8 characters (documented constant, mirrored on the Angular side per Product Brief Section 21).

Authorization rules:

/register and /login are public. Every other route in the app passes through authenticate.

Error cases:

Missing fields → 400. Duplicate email → 409. Wrong password / unknown email → 401 (generic message). Suspended account → 403.

Security considerations:

Never log the plaintext password, ever – not even at debug level. Never return passwordHash in any response (schema already sets select:false, but double-check the login response explicitly strips it if .select('+passwordHash') was used to fetch it).

Testing:

Register with valid data → 201 + token present.

Register with a duplicate email → 409.

Login with correct credentials → 200 + token.

Login with wrong password → 401, message does not reveal whether email existed.

Login as a suspended user → 403.

Call a protected route with no Authorization header → 401.

Call a protected route with an expired token → 401.

Expected result:

Every other engineer can now write `router.get('/something', authenticate, controller)` and trust req.user is a real, active, non-suspended user.

Common beginner mistakes:

Storing the JWT secret in code instead of .env. Returning different error messages for "email not found" vs "wrong password" (an information-leak). Forgetting to check req.user.status on every request (checking only at login time misses users suspended mid-session).

Handoff:

authenticate and a documented req.user shape are consumed by every other engineer's routes starting Phase 2.

Definition of Done:

All tests above pass; authenticate exported and documented with a one-paragraph usage comment at the top of the file; PR reviewed by one teammate; merged to develop.

TASK 1.B — RBAC / authorize Middleware

Owner: Engineer 1

Goal:

Build the generic `requireRole(...roles)` and `requireOwnership(getOwnerId)` middleware factories used across every domain.

Why this exists:

Section 07's permission matrix must be enforced identically everywhere — one bug in a hand-rolled check on one route should not be able to happen because there is no shared implementation to get right once.

Files to create:

`src/middleware/authorize.js`

Step 1 — `requireRole(...allowedRoles)`:

Returns an Express middleware. Reads `req.user.role` (already attached by `authenticate`, which must always run first). If not in `allowedRoles`, respond 403 with `{ success:false, error:{ code:'FORBIDDEN', message:"You don't have permission to do that." } }` (matches Product Brief Section 22's exact user-facing copy).

Step 2 — `requireOwnership(getOwnerId)`:

`getOwnerId` is a small function passed in per-route, e.g.
`(req) => req.resource.providerId` (after a prior middleware loaded the document) or an async function that loads the document by `req.params.id` and returns its owner field.
 Compares `String(ownerId) === String(req.user._id)`. Admins (`req.user.role === 'admin'`) always pass, short-circuiting the comparison.

Step 3 — Document the required middleware order for any route needing both:

`authenticate` → `requireOwnership(...)` (loads + checks the specific doc) → `validate(...)` → controller. For admin-only routes: `authenticate` → `requireRole('admin')` → controller.

Business rules:

A suspended user never reaches here (`authenticate` already blocked them).
 Ownership checks always run server-side, regardless of what the Angular UI hides.

Testing:

`requireRole` rejects a 'user' hitting an admin-only mock route → 403.
`requireRole` allows an 'admin'.
`requireOwnership` rejects a user editing someone else's mock resource → 403.
`requireOwnership` allows the actual owner.
`requireOwnership` allows an admin regardless of ownership.

Common beginner mistakes:

Checking ownership by comparing `req.user._id` to `req.body.providerId` (client-supplied, untrustworthy) instead of the stored document's field. Forgetting the `String()` coercion when comparing `ObjectId` to a string, causing false negatives.

Handoff:

Every other engineer imports `{ requireRole, requireOwnership }` — nobody re-implements this logic locally.

Definition of Done:

Unit tests pass; documented with a usage example in the file header; merged before Phase 2 routes are built by Engineers 2–4.

Remaining Engineer 1 tasks (same TASK format, condensed here — each still requires full write-ups when executed):

- **TASK 1.C** — `errorHandler` centralized middleware (Section 16 below defines its exact contract).
- **TASK 1.D** — `GET/PUT /api/users/me` (profile read/update; self-only via `requireOwnership`).
- **TASK 1.E** — `POST /api/organizations` (register org, `ownerUserId = req.user._id`, status defaults `pending`).
- **TASK 1.F** — `GET/PUT /api/organizations/:id` (public read; update restricted to owner or admin).
- **TASK 1.G** — `POST /api/organizations/:id/verify` (admin-only; body `{ decision: 'approved'|'rejected', rejectionReason? }`; calls a table-driven `organizationService.transitionVerification` mirroring the DB plan's Section 9.6 table — reject requires a reason, enforced in the service, not scattered in the controller).
- **TASK 1.H** — Admin user management: `GET /api/admin/users`, `PUT /api/admin/users/:id/suspend`, `PUT /api/admin/users/:id/reactivate` (admin-only; never a hard delete, per DB plan Section 12.5).

Dependencies: none (Phase 1 foundation). Handoff: `authenticate`, `authorize`, `errorHandler`, `User`, `Organization` merged before any other engineer starts Phase 2.

09 — Engineer 2 Plan — ادهم محمد حسن عبد السميع

Mission: Supply. Owns `categories`, `resources`, and everything a provider needs to publish and manage surplus.

Prerequisite reading: DB plan Sections 6.3, 6.4, 8.3, 8.4, 9.2 (Resource state machine — already fully specified as a table-driven `resourceLifecycleService`, built in the DB phase; this plan wires it to HTTP).

TASK 2.A — Resource CRUD Routes + Ownership Enforcement

Owner: Engineer 2

Goal:

Expose resources as a REST resource: list/search, create, read, update, and status-based delete, all authorization-correct.

Why this exists:

This is the "supply" half of the core product loop (FR-005, FR-006, FR-022).

Prerequisites:

Resource model + resourceLifecycleService already built (DB plan Task 2.2). authenticate / authorize available (Engineer 1, Phase 1).

Files to create:

src/controllers/resources.controller.js

src/routes/resources.routes.js

src/validators/resource.validators.js

Step 1 – GET /api/resources (public):

Reads query params category, city, area, status (default to browsable statuses: published/available), page, limit. Builds a Mongoose filter object explicitly – never pass req.query straight into .find() (NoSQL injection risk, Product Brief Section 13/DB plan Section 13 "injection prevention" – validate each param is a string/known enum before use). Uses the { status, categoryId, createdAt } index (DB plan Section 10). Returns paginated results via the shared pagination util.

Step 2 – POST /api/resources (authenticated):

Body validated against resource.validators.js (category, quantity>0, location, availability window with end>start – same rule the schema also enforces, so a bad request fails fast with a clear 400 instead of surfacing a raw Mongoose CastError). providerId always set from req.user._id server-side, never from the body. If providerOrgId is supplied, verify (a) it belongs to req.user (organizationService.isOwner) and (b) the org is approved – otherwise 403. Created in status: 'draft'.

Step 3 – PUT /api/resources/:id (owner or admin):

Loads the resource first (a small middleware, loadResource, attaches req.resource), then requireOwnership(req => req.resource.providerId). Only allows editing fields that make sense pre-lifecycle-completion (reject edits to a completed/impact_recorded resource with a 409 "This listing can no longer be edited"). Status changes go through a separate mechanism (Step 4), not this generic update.

Step 4 – PUT /api/resources/:id/status (owner or admin):

Body: { action: 'publish' | 'markUnavailable' | 'cancel' | ... }. Calls resourceLifecycleService.transitionResource(resource, action, req.user) – the ONE place status changes happen. Never allow a raw { status: 'completed' } write from the client (DB plan Section 12.2's core rule).

Step 5 – DELETE /api/resources/:id (owner or admin):

Not a hard delete – internally calls the same transition service with a 'cancel' action (DB plan Section 12.5). Documented clearly in the endpoint's Postman entry so nobody expects a 204-with-no-trace.

Business rules:

A provider cannot publish on behalf of an organization they don't own or that isn't verified.

A resource in a terminal state (completed, impact_recorded, cancelled) cannot be edited.

Validation rules:

Per DB plan 8.4 (title 3–100 chars, description ≤1000, quantity>0, availability window end>start, category must exist and be active).

Authorization rules:

Read: public. Write: authenticated + owner (or admin). Status transitions: owner (or admin), enforced additionally by the transition table itself (some actions are system-only, e.g. 'expire' – reject if a non-system actor requests it).

Error cases:

Invalid category → 400 "Category not found or inactive". Editing someone else's resource → 403. Editing a terminal-state resource → 409. Invalid transition (e.g. completed→available) → 409 with the specific message from the lifecycle service.

Security considerations:

Query params sanitized before use in `.find()` filters (never ``Resource.find(req.query)`` directly). `ObjectId` params validated with the `objectId.js` util before any lookup – an invalid format should return 400, not crash the process with a `CastError`.

Testing:

Full CRUD happy path. Ownership rejection (403) for edit/delete by a non-owner non-admin. Every valid and invalid lifecycle transition from DB plan Section 9.2's matrix. Category filter + location filter queries return correct, indexed results (verify with `.explain()` in a dev script, not in the automated suite).

Expected result:

A provider can list, publish, update, and retire a resource entirely through the API, with every transition backend-enforced regardless of what the Angular UI allows.

Common beginner mistakes:

Allowing `PUT /api/resources/:id` to silently accept a status field and write it directly (bypasses the lifecycle table entirely – the single most important rule in this task). Trusting `req.query` values without validating their shape.

Handoff:

`resourceLifecycleService.transitionResource` is reused by Engineer 4's `handoverService` for the cancel/no-show cascade (Section 12 below) – Engineer 4 imports and calls it, never re-implements resource status logic.

Definition of Done:

See Section 24. Additionally: a teammate has manually walked the full lifecycle via Postman.

Remaining Engineer 2 tasks (condensed):

- **TASK 2.B** — `GET/POST/PUT/DELETE /api/categories` (admin-only writes; public reads; `isActive` soft delete, never hard delete — DB plan Section 12.5).
- **TASK 2.C** — Search/filter query-building helper shared by `GET /api/resources` and (read-only, consumed by Engineer 3) the candidate-retrieval query used inside `matchingService.generateMatches`.

Dependencies: `User` model (Engineer 1). Handoff: `resources / categories` merged + `resourceLifecycleService` exposed before Engineer 3 starts the matching engine (Phase 2 → Phase 3 gate).

10 — Engineer 3 Plan — دينا عبدالله عليان محمد

Mission: Demand & Matching. Owns `requests`, `matches`, and the explainable rule-based scoring engine — the product's flagship feature.

Prerequisite reading: DB plan Sections 6.5, 6.6, 8.5, 8.6, 9.3, 9.4, Section 12 of the Product Brief (scoring model).

TASK 3.A — Request CRUD Routes

Owner: Engineer 3

Goal:

Mirror Engineer 2's resource CRUD pattern for requests — same shape, same rigor, applied to the demand side.

Steps (condensed — follow the exact structure of TASK 2.A):

1. GET `/api/requests` (public/authenticated per team decision — Product Brief doesn't restrict browsing requests).
2. POST `/api/requests` — `requesterId` from `req.user`, `requesterOrgId` ownership+verification check identical to Engineer 2's Step 2.
3. PUT `/api/requests/:id` — owner or admin, blocked on terminal states.
4. PUT `/api/requests/:id/status` — routes through `requestLifecycleService.transitionRequest` only (DB plan Section 9.3's table).
5. DELETE `/api/requests/:id` — status-based cancellation, not hard delete.

Business/validation/authorization/error rules: identical structure to TASK 2.A, values per DB plan 8.5 (urgency enum, `quantity>0`, category must exist).

Testing: identical structure to TASK 2.A, exercised against DB plan Section 9.3's transition matrix instead of 9.2's.

Common beginner mistakes: copying Engineer 2's `resourceLifecycleService` by copy-paste-and-rename instead of writing `requestLifecycleService` against the request-specific table — the PATTERN is shared, the DATA (transition table) is not.

TASK 3.B — Matching Engine HTTP Surface

Owner: Engineer 3

Goal:

Expose `matchingService` (already built in the DB phase, DB plan Task 3.2) as
 POST `/api/matches/:resourceId/generate`, plus GET `/api/matches`, PUT `/api/matches/:id/accept`,
 PUT `/api/matches/:id/reject`.

Why this exists:

FR-009–FR-011: compute scores, present ranked candidates, let either side accept/reject.

Prerequisites:

`matchingService.calculateScore / generateMatches` already built and unit-tested (DB plan Task 3.2). Resource must be in status: 'available' before matches can be generated against it.

Step 1 – POST `/api/matches/:resourceId/generate`:

Authorization: resource owner, or a system/cron caller (documented distinctly – for the MVP demo this can simply be owner-or-admin triggered, since a scheduled job is explicitly P2).

Loads the resource, verifies status === 'available', calls
`matchingService.generateMatches(resourceId)`, returns the ranked list with each match's
`scoreBreakdown`.

Step 2 – GET `/api/matches`:

Authenticated. Filters to matches where `req.user._id === providerId OR requesterId` (uses the denormalized fields from DB plan Section 07.4 – this is exactly why they're denormalized: a single indexed query, no join). Admins may pass a query param to see all.

Step 3 – PUT `/api/matches/:id/accept`:

This is the ONE cross-domain, transactional endpoint in the whole system (Section 13 below).
 Authorization: requireOwnership checking BOTH `matches.providerId` and `matches.requesterId` (either party may accept their side – clarify with the team whether both must independently accept or a single accept is sufficient per the Product Brief's FR-011 wording; document whichever is chosen, since the brief allows either reading and the schema (single status field) implies single-accept is the MVP interpretation – do not silently assume without recording the decision here).

Calls `matchingService.acceptMatch(matchId, actingUser)` which opens a MongoDB session and:

- `matches.status: 'proposed' → 'accepted'`
- `resources.status: transitions via resourceLifecycleService (available → matched → accepted, or directly per the table)`
- `requests.status: transitions via requestLifecycleService similarly`
- calls `handoverService.createHandoverForMatch(match, session)` – Engineer 4's function, imported, never re-implemented – to create the handovers document in the SAME transaction.

If any step fails, `session.abortTransaction()` – nothing partially applies.

Step 4 – PUT `/api/matches/:id/reject`:

Single-document-ish operation: `matches.status → 'rejected'`, resource released back to available via `resourceLifecycleService` (no transaction required – see Section 13).

Business rules:

A match cannot be generated against a resource that isn't available.

The partial unique index (DB plan Section 6.6) prevents two simultaneous proposed matches for the same (`resourceId`, `requestId`) – the service should catch the resulting duplicate-key error and translate it into a clean 409, not let a raw Mongo error leak to the client.

Validation rules:

`:resourceId` and `:id` must be valid ObjectIds (400 if not, via the `objectId.js util`) before any query runs.

Authorization rules:

Generate: resource owner or admin. Accept/Reject: party to the match, or admin.

Error cases:

Resource not available → 409. Duplicate proposed match → 409 (translated from the Mongo unique-index error). Non-party attempting accept/reject → 403. Match already accepted/rejected/expired → 409 "This match is no longer pending."

Security considerations:

No sensitive contact info is exposed in the match list response until the match is accepted – providerId/requesterId are just IDs; full contact details are a separate, later concern (handoverService/contactInfo, per Product Brief's privacy note) and must not be joined into this response by accident via an over-eager .populate().

Testing:

Generate against a seeded resource/request pair shaped to score above threshold – confirm a Match is created with a populated scoreBreakdown.

Attempt to generate a second proposed match for the same pair – confirm 409.

Accept as a non-party – confirm 403.

Accept happy path – confirm matches.status, resources.status, requests.status, and a new handovers document all update together.

Kill the transaction mid-way in a test (e.g. mock handoverService.createHandoverForMatch to throw) – confirm NONE of the three writes persisted (this is the single most important test in the entire backend – it proves the transaction boundary actually works).

Expected result:

An explainable, tested matching engine reachable over HTTP, with match acceptance guaranteed atomic across three collections.

Common beginner mistakes:

Implementing accept as three separate, un-transactioned .save() calls "because it's simpler" – this is exactly the inconsistency Section 13 exists to prevent. Forgetting to pass the session through to EVERY write inside the transaction (a write without { session } silently runs outside the transaction and defeats the whole point).

Handoff:

matchingService.acceptMatch() is the ONLY entry point that creates a handover – Engineer 4 exposes handoverService.createHandoverForMatch(match, session) and does not call it from anywhere else. This contract is restated in Section 12.

Definition of Done:

All tests above pass, including the forced-rollback test; PR jointly reviewed by Engineer 3 and Engineer 4 (this endpoint touches both domains – DB plan Section 25 calls this out explicitly).

Dependencies: User, Resource, Category (Engineers 1-2). Handoff: matchingService and the match-accept transition are consumed by Engineer 4's handover creation (Phase 3 → Phase 4 gate).

11 — Engineer 4 Plan — عبدالستار محمد عبدالستار

مطالع

Mission: Transfer, Trust & Impact. Owns handovers, reports, notifications, contributions, and — per the Product Brief's own team structure (Section 33) — cross-module integration testing and end-to-end consistency.

Prerequisite reading: DB plan Sections 6.7–6.10, 8.7–8.10, 9.5, 9.7, 15 (transactions).

TASK 4.A — Handover Confirmation Endpoint

Owner: Engineer 4

Goal:

Expose `handoverService` (built in the DB phase) as
 POST `/api/transactions/:matchId/confirm`, implementing FR-013's two-sided guarantee at the API layer.

Why this exists:

The Product Brief's central trust mechanism: a transfer must never be completable by one side acting alone. This endpoint is where that promise is either kept or broken.

Prerequisites:

`handoverService.confirm(handoverId, side, userId)` already built and tested (DB plan Task 4.1). Engineer 3's match-accept flow already creates the handovers document.

Step 1 – Route + controller:

POST `/api/transactions/:matchId/confirm`. Looks up the handovers document by `matchId` (unique index, DB plan Section 8.7) – NOT by a handover ID in the URL, matching the Product Brief's API design (Section 20) exactly.

Step 2 – Determine "side":

Compare `req.user._id` against the loaded handover's `providerId` and `seekerId`. If it matches neither, 403. If it matches `providerId`, `side='provider'`; if `seekerId`, `side='seeker'`. Never accept a `side` field from the request body – it is derived server-side from who is authenticated, which is the entire point of the two-sided guarantee.

Step 3 – Call `handoverService.confirm(handover._id, side, req.user._id)`:

Sets the corresponding confirmation flag idempotently (a second confirm from the same side is a 200 no-op, not an error – Section 4.1 of the DB plan). If both flags are now true, the service internally: sets `status:'completed'`, `completedAt`, calls `contributionService.recordCompletedTransfer(handover)`, and transitions the underlying resource/request to their completed states via the lifecycle services (imported, not reimplemented).

Step 4 – Response:

Return the updated handover status. If status is now `'completed'`, the response should make that unambiguous (e.g. `{ status: 'completed', bothConfirmed: true }`) so the frontend can show a clear success state (Product Brief Section 23, "never a silent success").

Business rules:

No endpoint anywhere in the API accepts a raw `{ status: 'completed' }` write to handovers. The ONLY way status becomes completed is both sides independently calling this endpoint.

Validation rules:

`:matchId` must be a valid `ObjectId`. A handover must exist for that match (if a client calls this before match acceptance, 404 "No handover found for this match").

Authorization rules:

Only the two named parties (`providerId`, `seekerId`) on that specific handover may confirm – not any authenticated user, not even other admins by default (an admin override, if desired, would be a distinct, explicitly-logged endpoint – not folded into this one, to avoid weakening the two-sided guarantee).

Error cases:

Non-party confirms → 403. Handover already cancelled/no_show → 409 "This handover is no longer active." Confirming twice from the same side → 200, no state change (idempotent, not an error – a double-tap in the UI must not look like a failure to the user).

Security considerations:

This is the single highest-trust write path in the system – log every confirm call (who, when, which side) at the application log level (Section 17 below) for auditability,

without logging any sensitive contact info.

Testing:

Provider confirms alone → status stays `in_progress`, only `confirmedByProvider` true.

Seeker then confirms → status becomes `completed`, `completedAt` set, a `Contribution` document exists, `users.stats` incremented for both parties.

A THIRD user (not provider or seeker on this handover) attempts to confirm → 403.

Directly manipulate the document in a test to set both flags via a raw update bypassing the service, then confirm the service's own read-modify-write logic doesn't double-fire the contribution creation on a subsequent unrelated call (protects the "exactly one Contribution per completed handover" invariant from DB plan Section 27's checklist).

Confirm the same side twice – verify no duplicate timestamp overwrite, no error.

Expected result:

FR-013 is enforced as a structural guarantee, not a UI convention – verified by a test that attempts to break it and fails to.

Common beginner mistakes:

Accepting a side field from the client instead of deriving it from `req.user._id` (this alone would completely defeat the two-sided guarantee – flag it explicitly in code review).

Forgetting the idempotency requirement and throwing a 409 on a double-confirm, which reads as a bug to end users.

Handoff:

None outward – this is a terminal endpoint for the transfer lifecycle. Inward: consumes Engineer 2's and Engineer 3's lifecycle services for the resource/request side-effects.

Definition of Done:

All tests pass including the non-party-rejection and idempotency tests; a teammate has manually attempted (and failed) to complete a transfer from one side only, via Postman, as a sanity check before sign-off.

Remaining Engineer 4 tasks (condensed — same TASK format required at execution time):

- **TASK 4.B** — `POST /api/reports`, `GET /api/reports` (admin), `PUT /api/reports/:id/resolve` (admin): polymorphic `targetType` / `targetId` validated against the real collection before save (`reportService.validateReportTarget` , DB plan Task 4.2) — a report against a non-existent target must fail with 400, not silently save a dangling reference.
- **TASK 4.C** — `notificationService.notify(...)` write-side integration only, called from Engineer 3's `matchingService` and this domain's `reportService` on resolution; no read/mark-as-read endpoints in the MVP scope (P1 per DB plan Section 6.9 and Product Brief Section 10) — but the collection and write path ship now.
- **TASK 4.D** — `contributionService.recordCompletedTransfer` / `getContributionHistory` / `rebuildStatsCache` , exposed read-only via `GET /api/users/me/contributions` (or folded into the profile endpoint — team's call, document the decision) for FR-018.
- **TASK 4.E** — `GET /api/admin/analytics` — the aggregation pipelines from DB plan Section 14, admin-only, read-only, no new collection created for "analytics."
- **TASK 4.F (cross-cutting)** — Own the end-to-end integration test suite (Section 26 of the Product Brief's demo journey) and the final pre-demo QA pass (Section 21 below), consistent with this engineer's Product-Brief-assigned role as UX/Admin/Integration lead.

Dependencies: `Match` (Engineer 3), `resourceLifecycleService / requestLifecycleService` (Engineers 2–3, for cancel/no-show cascades). Handoff: `handoverService.confirm` is the only function any route calls for confirmation — no other engineer re-implements it.

12 — Cross-Engineer Contracts

Ownership never moves between engineers without updating this document. Where two domains must interact, the contract is explicit, not implied:

From	Contract Function	Consumed By	Rule
Eng 2 → Eng 3	<code>resourceLifecycleService.transitionResource(resource, action, actor)</code>	<code>Match</code> generation/acceptance, handover cancel cascade	Eng 3/4 call it; never write <code>resource.status</code> directly
Eng 3 → Eng 3 (self)	<code>matchingService.acceptMatch(matchId, actor)</code>	The <code>PUT /api/matches/:id/accept</code> controller	Wraps the 3-collection transaction; the only place a match is accepted
Eng 3 → Eng 4	<code>handoverService.createHandoverForMatch(match, session)</code>	Called only from inside <code>matchingService.acceptMatch</code> 's transaction	Eng 4 exposes it; Eng 3 calls it; Eng 4 never creates a handover any other way
Eng 4 → Eng 2/3	<code>resourceLifecycleService / requestLifecycleService cancel/no-show transitions</code>	<code>handoverService cancel/no_show</code> paths	Eng 4 calls the owning engineer's transition function; never edits <code>resources / requests</code> status fields itself
Eng 4 → Eng 3	<code>notificationService.notify(...)</code>	Called from <code>matchingService.generateMatches (match_created)</code>	Eng 3 calls Eng 4's exported function; Eng 4 never reaches into <code>matches</code>
Eng 1 → all	<code>authenticate, authorize.requireRole/requireOwnership, errorHandler</code>	Every route file	No engineer re-implements auth/authz locally

Any change to a function signature listed above requires posting in the team channel and updating this table **before** merging — mirrors the DB plan's Section 18 Shared Database Contract process, extended to service-layer contracts.

13 — Transactions

Directly extends DB plan Section 15 with the API-layer implication for each case.

Operation	Documents touched	Transaction?	Why
Match acceptance	<code>matches</code> , <code>resources</code> , <code>handovers</code> (+ <code>requests</code>)	Yes	Three-to-four collections change together; a half-applied accept (match shows accepted, resource stuck at matched, no handover) is exactly the inconsistency the product's trust model cannot tolerate
Match rejection	<code>matches</code> , <code>resources</code>	No	Two independent-enough writes; worst case of a rare partial failure is a resource stuck <code>matched</code> instead of <code>available</code> — recoverable via an admin/cron sweep, not worth transaction overhead for MVP scale
Handover confirmation (single side)	<code>handovers</code> (one document)	No	Single-document atomicity is native to MongoDB
Handover completion (both sides true)	<code>handovers</code> , <code>contributions</code> , <code>users.stats</code> (×2)	Recommended, not required	Small transaction wrapping the ledger insert + cache increment avoids drift; if skipped for MVP speed, <code>rebuildStatsCache</code> (DB plan Task 4.4) is the documented recovery path
Report creation/resolution	<code>reports</code> (one document)	No	Single-document write

How the match-acceptance transaction must be implemented (Node/Mongoose):

```

const session = await mongoose.startSession();
try {
  session.startTransaction();
  await Match.updateOne({ _id }, { status: 'accepted' }, { session });
  await resourceLifecycleService.transitionResource(resource, 'accept', actor, { session });
  await requestLifecycleService.transitionRequest(request, 'accept', actor, { session });
  const handover = await handoverService.createHandoverForMatch(match, session);
  await session.commitTransaction();
  return handover;
} catch (err) {
  await session.abortTransaction();
  throw err;
} finally {
  session.endSession();
}

```

Every write inside the `try` block must pass `{ session }` — a write missing it silently executes outside the transaction, which defeats the entire guarantee. This is called out as a named "common beginner mistake" in Task 3.B and must be specifically checked in code review for this one endpoint.

What NOT to do: do not wrap read-only aggregation queries in a session; do not wrap single-document updates; do not use transactions "defensively" everywhere — each additional transaction adds latency and MongoDB replica-set overhead the MVP's scale does not need (DB plan Section 15's closing point, restated).

14 — API Specification

Full endpoint-to-database mapping already exists in the DB plan (Section 17). This section adds authentication/authorization/validation/error detail per endpoint. Format: `METHOD path - auth - role/owner - key validation - success - key errors`.

Identity & Organizations (Eng 1)

POST	/api/auth/register	– public	–	– email+password	– 201 –
400,409					
POST	/api/auth/login	– public	–	– email+password	– 200 –
400,401,403					
POST	/api/auth/logout	– auth	–	–	– 200 – 401
GET	/api/users/me	– auth	– self	–	– 200 – 401
PUT	/api/users/me	– auth	– self	– profile fields	– 200 –
400,401					
POST	/api/organizations	– auth	– self-owner	– org schema	– 201 –
400,401					
GET	/api/organizations/:id	– public	–	– objectId	– 200 – 404
PUT	/api/organizations/:id	– auth	– owner/admin	– org schema	– 200 –
400,403,404					
POST	/api/organizations/:id/verify	– auth	– admin	– decision+reason*	– 200 –
400,403,404					
GET	/api/admin/users	– auth	– admin	– pagination	– 200 – 403
PUT	/api/admin/users/:id/suspend	– auth	– admin	– objectId	– 200 –
403,404					

*reason required only when decision === 'rejected'

Supply (Eng 2)

GET	/api/categories	– public	–	–	– 200 – –
POST	/api/categories	– auth	– admin	– name unique	– 201 –
400,403,409					
PUT	/api/categories/:id	– auth	– admin	– name unique	– 200 –
400,403,404					
DELETE	/api/categories/:id	– auth	– admin	– (soft)	– 200 –
403,404					
GET	/api/resources	– public	–	– query params	– 200 – –
POST	/api/resources	– auth	– self-owner	– resource schema	– 201 –
400,401,403					
GET	/api/resources/:id	– public	–	– objectId	– 200 – 404
PUT	/api/resources/:id	– auth	– owner/admin	– resource schema	– 200 –
400,403,404,409					
PUT	/api/resources/:id/status	– auth	– owner/admin	– action enum	– 200 –
403,404,409					
DELETE	/api/resources/:id	– auth	– owner/admin	– (status-based)	– 200 –
403,404					

Demand & Matching (Eng 3)

GET	/api/requests	– auth	–	– query params	– 200 – 401
POST	/api/requests	– auth	– self-owner	– request schema	– 201 –
400,401,403					
PUT	/api/requests/:id	– auth	– owner/admin	– request schema	– 200 –
400,403,404,409					
PUT	/api/requests/:id/status	– auth	– owner/admin	– action enum	– 200 –
403,404,409					
DELETE	/api/requests/:id	– auth	– owner/admin	– – (status-based)	– 200 –
403,404					
GET	/api/matches	– auth	– self (party)	– –	– 200 – 401
POST	/api/matches/:resourceId/generate	– auth	– resource owner/admin	– resource available	– 201 –
403,404,409					
PUT	/api/matches/:id/accept	– auth	– party to match	– objectId	– 200 –
403,404,409					
PUT	/api/matches/:id/reject	– auth	– party to match	– objectId	– 200 –
403,404,409					

Transfer, Trust & Impact (Eng 4)

POST	/api/transactions/:matchId/confirm	– auth	– party to handover	– –	– 200 –
403,404,409					
POST	/api/reports	– auth	–	– polymorphic target	– 201 –
400,401,404					
GET	/api/reports	– auth	– admin	– pagination	– 200 – 403
PUT	/api/reports/:id/resolve	– auth	– admin	– resolution string	– 200 –
400,403,404					
GET	/api/users/me/contributions	– auth	– self	– –	– 200 – 401
GET	/api/admin/analytics	– auth	– admin	– –	– 200 – 403

Every list endpoint (`GET` without `:id`) uses the shared `?page=1&limit=20` convention from the DB plan's Shared Database Contract (Section 18 of that document) — no engineer invents a different pagination shape.

15 — Validation Strategy

Two enforcement layers, matching Product Brief Section 21 exactly:

- Frontend (Angular Reactive Forms)** — immediate feedback; never trusted by the backend.
- Backend (mandatory, the only trusted layer)** — two sub-layers of its own:
 - Request-payload validators** (`src/validators/*.js`) run inside the `validate(schema)` middleware *before* the controller — catch shape errors early with field-level 400s (e.g. "quantity must be a positive number") rather than letting a malformed payload reach a service function.
 - Mongoose schema validation** — the final backstop at the persistence layer (already specified per collection in DB plan Section 08); catches anything a request

validator missed or that arrives via a path other than the standard controllers (e.g. a seed script bug).

Rule for every write endpoint: **validate** → **authorize** → **business logic**, in that order, so a request that is both malformed and unauthorized returns the validation error (cheaper to compute, and doesn't leak whether the target resource exists to an unauthorized caller in ownership-check order — a minor but real information-leak consideration).

Never trust `req.query` values passed into a Mongoose filter unchecked — every query param used in a `.find()` must be validated against an allow-list of expected keys/types first (NoSQL-injection prevention, DB plan Section 13 and Product Brief NFR-004).

16 — Error Handling

One centralized `errorHandler` Express middleware (Engineer 1, Task 1.C), mounted **last**, after all routes. Every controller either lets a thrown error propagate (`next(err)` or an async-wrapper util) or throws a typed `AppError` — no controller formats its own error response.

Response shape (fixed, matches both source documents):

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "This listing no longer exists."
  }
}
```

Error type	HTTP status	Example code	User-facing message
Validation	400	VALIDATION_ERROR	field-level detail, e.g. "Quantity must be greater than 0."
Authentication	401	UNAUTHENTICATED	"Please log in to continue."
Authorization	403	FORBIDDEN	"You don't have permission to do that."
Not found	404	NOT_FOUND	"This listing no longer exists."
Conflict / business rule	409	CONFLICT	e.g. "This request has already been matched."

Error type	HTTP status	Example code	User-facing message
Server error	500	INTERNAL_ERROR	"Something went wrong. Please try again." (generic — full detail logged server-side only, per NFR-012)

Mongoose `ValidationError` and `CastError` are caught centrally and mapped to `VALIDATION_ERROR` /400 (a malformed ObjectId in a URL param becomes a clean 400, not a raw stack trace). The duplicate-key error (code: 11000) from a unique index is mapped to `CONFLICT` /409 with a friendly message per field (e.g. "An account with this email already exists.").

17 — Security

Checklist, proportional to a graduation-project MVP (DB plan Section 13, extended):

- Passwords hashed with `bcrypt` ; `passwordHash` never returned in any API response.
- JWT secret and `MONGODB_URI` only in `.env` , never committed; `.env.example` has placeholders only.
- Every write endpoint validates and sanitizes input (Section 15); no raw `req.query` / `req.body` passed into a Mongoose query.
- Ownership checked server-side on every mutating endpoint touching a `providerId` / `requesterId` / `ownerUserId` / `reporterId` -bearing document — never inferred from the client.
- ObjectId route params validated before use; invalid format → 400, not a crash.
- Rate limiting on `POST /api/resources` , `POST /api/requests` , `POST /api/reports` (a simple `express-rate-limit` per-IP or per-user window is sufficient — matches Product Brief's "abuse prevention" requirement without building custom infrastructure).
- CORS configured to the known Angular origin(s) only, not `*` , once a deployment origin is known.
- Basic HTTP security headers via `helmet()` .
- Error responses never leak stack traces, internal file paths, or raw Mongo error text to the client (Section 16).
- `organizations.contactInfo` is only included in API responses after the relevant match has reached `accepted` status — enforced by the controller/service response shaping, not by the schema (DB plan Section 13's privacy note, restated as an API-layer obligation here since it's easy to accidentally leak via an eager `.populate()`).

- No national ID, payment, or other high-sensitivity fields exist anywhere in this system — by design.

18 — Testing Strategy

Builds directly on DB plan Section 27 (data-layer tests, already specified) plus the API-layer tests this document adds:

Layer	What	Tooling
Unit	<code>matchingService.calculateScore</code> , every lifecycle transition table, <code>authService</code> hashing/token logic	Jest/Mocha, no DB needed (or an in-memory Mongo for model-adjacent unit tests)
Integration (service)	Transaction rollback test for <code>matchingService.acceptMatch</code> (Section 13's critical test), <code>handoverService.confirm</code> idempotency and non-party rejection	Jest/Mocha + a real or in-memory test MongoDB
API	Every endpoint in Section 14 — one happy-path test + at least one negative test per row	<code>supertest</code>
Authorization	Every role-restricted and ownership-restricted endpoint rejects the wrong actor	<code>supertest</code>
Validation	Every validation rule in Section 15 — missing fields, bad enums, boundary values	<code>supertest</code>
Negative/edge cases	Duplicate registration, wrong password, suspended user, invalid ObjectId, nonexistent reference, invalid lifecycle transition, duplicate proposed match, one-sided handover completion, double confirmation, invalid report target, resolving a report without a resolution string	<code>supertest</code>

Representative required test scenarios (directly from the Product Brief, Section 35, mapped to this plan):

- A provider publishes a resource with missing quantity → 400 with a field-level message.

- A seeker attempts to edit another user's request → 403.
- A match is accepted-confirmed by only one party → handover stays `in_progress`, never `completed`.
- An unverified organization attempts to publish a standing request → blocked (409 or 403, team's choice, documented) until verification is approved.

Each engineer owns the tests for their own routes/services; Engineer 4 additionally owns the full integration test that runs Section 21's end-to-end flow start to finish.

19 — Seed Data

Adopts DB plan Section 16 directly (seed script content, `NODE_ENV` production guard, demo-shaped resource/request pair, one pre-completed handover+contribution for non-zero dashboard numbers on first load). This plan adds only the API-layer seed requirement: **at least one valid JWT-issuable seeded account per role** (`admin`, `provider`, `seeker`, `org owner`) with known, documented (non-production) credentials in `seed/README.md`, so the live demo can log in as a pre-seeded user without registering on stage.

20 — Git Workflow

Adopts the DB plan's Section 24 branching/PR rules directly, extended to cover route/controller/service files (which now also carry per-engineer ownership, per Section 04's file tree):

- Branches: `feature/api/identity`, `feature/api/resources`, `feature/api/matching`, `feature/api/trust-impact` — one per engineer, layered on top of that engineer's already-merged `feature/database/*` branch.
- Commit convention: `api(<domain>): <what changed>` — e.g. `api(matching): add transactional accept endpoint`.
- PRs required for every merge to `develop`; a PR touching another engineer's route/controller/service file requires that engineer's explicit approval.
- The match-acceptance PR (Task 3.B) and the handover-confirmation PR (Task 4.A) are **jointly reviewed** by Engineers 3 and 4, since they form the one true cross-domain contract in the system (Section 12).
- Any change to a shared middleware file (`authenticate.js`, `authorize.js`, `errorHandler.js`) is Engineer 1's to make; other engineers request changes via issue, not direct edits.

21 — Integration Plan

All four `feature/api/*` branches merge to `develop` in dependency order (Section 26), then the team runs, in a shared dev database seeded per Section 19:

```
Register (all 4 roles) → Login → Create Organization → Admin Approves Org
  → Create Resource → Publish Resource
  → Create Request → Publish Request
  → Generate Match → Accept Match (transaction) → Handover created
  → Provider Confirms → Seeker Confirms → Handover Completed → Contribution Created
  → Reputation/Impact cache updated → Resource → impact_recorded
Report User / Resource / Request → Admin Reviews → Report Resolved → Notification Created
```

This is the API-layer execution of the DB plan's Section 28 end-to-end scenario — same steps, now driven through HTTP endpoints instead of direct model calls, run via the integration test suite (Section 18) and manually via Postman before the live demo.

Additional integration checks specific to the API layer: every response shape matches Section 16's contract; every list endpoint's pagination is consistent; CORS allows the actual frontend origin; JWTs issued in Phase 2 still authenticate correctly against routes built in Phase 5 (a regression that only shows up once all domains are merged together).

22 — Deployment Preparation

Scoped to what a graduation demo actually needs — not a production rollout:

- `.env.example` complete and accurate (`MONGODB_URI` , `MONGODB_URI_TEST` , `JWT_SECRET` , `JWT_EXPIRES_IN` , `PORT` , `NODE_ENV`).
- `npm start` (production-mode entry via `server.js`) and `npm run dev` (with a watcher) both work from a clean `git clone` .
- CORS origin configurable via env var, so the same build works locally and against a deployed frontend.
- A single `README.md` at the repo root: setup steps, seed instructions, and a link to the API docs (Section 23).
- `NFR-010` ("structured to allow horizontal scaling") is satisfied by design (stateless JWT auth, no in-memory session state, no local file storage) — no additional infrastructure work is required to meet it for the MVP.
- No secrets in source control — verified by a final `git log -p` scan of `.env` -shaped content before the repo is shared with evaluators.

23 — Documentation

- A Postman collection (or lightweight OpenAPI/Swagger YAML, team's choice — "prefer OpenAPI if lightweight") covering every endpoint in Section 14: request shape, auth requirement, example success response, example error responses. Exported/committed to `docs/` so evaluators can exercise the API without reading source.
 - Each service file (`*.js` under `src/services/`) carries a header comment: purpose, owning engineer, and which other services it calls into or is called by (mirrors Section 12's contract table so the contract is discoverable from the code itself, not only from this document).
 - This document (`DAWWARHA - BACKEND IMPLEMENTATION PLAN`) and the `MongoDB Implementation Plan` together are the team's single source of truth; any deviation discovered during implementation is corrected in the relevant document, not left undocumented in someone's branch.
-

24 — Definition of Done

Per task: code + validation + authorization + tests (positive and negative) + documentation (Postman entry) + reviewed by a teammate + no known regressions in already-merged behavior.

Per engineer: every task in their section (08–11) merged to `develop` ; their domain's tests pass in isolation; their handoff functions (Section 12) are exported, documented, and demonstrably called correctly by the consuming engineer.

Per phase (Section 26): every engineer scheduled in that phase has met the "per engineer" bar; the phase's exit criteria (below) are met; `develop` builds and boots cleanly with the seed script run against it.

For the entire backend: the full integration scenario (Section 21) runs start to finish without manual data patching; every item in Section 25's checklist is checked; the demo flow (Section 28) has been rehearsed at least once against the merged `develop` branch.

A task is never "done" because it compiles. It is done when the checklist above is fully satisfied.

25 — Complete Task Checklist

PHASE 0 – Setup

- [] Repo initialized, .gitignore, .env.example, package.json, README skeleton
- [] database.js connection util (Eng 1, Task 1.1 – from DB plan)

PHASE 1 – Identity Foundation

- [] User model (DB plan Task 1.2)
- [] Organization model (DB plan Task 1.3)
- [] authService + authenticate middleware (Task 1.A)
- [] authorize middleware (Task 1.B)
- [] errorHandler middleware (Task 1.C)
- [] /api/auth/* routes
- [] /api/users/me routes (Task 1.D)
- [] /api/organizations/* routes + verify endpoint (Tasks 1.E–1.G)
- [] Admin user management routes (Task 1.H)

PHASE 2 – Supply & Demand (parallel once Phase 1 merges)

- [] Category model + routes (DB plan Task 2.1, Task 2.B)
- [] Resource model + resourceLifecycleService (DB plan Task 2.2)
- [] Resource CRUD + status routes (Task 2.A)
- [] Request model + requestLifecycleService (DB plan Task 3.1)
- [] Request CRUD + status routes (Task 3.A)

PHASE 3 – Matching

- [] Match model + matchingService (DB plan Task 3.2)
- [] Generate/list/accept/reject routes incl. transaction (Task 3.B)

PHASE 4 – Transfer, Trust, Impact

- [] Handover model + handoverService (DB plan Task 4.1)
- [] Confirm endpoint (Task 4.A)
- [] Report model + report routes (DB plan Task 4.2, Task 4.B)
- [] Notification model + write-side integration (DB plan Task 4.3, Task 4.C)
- [] Contribution model + contributionService + read endpoint (DB plan Task 4.4, Task 4.D)
- [] Admin analytics endpoint (Task 4.E)

PHASE 5 – Integration

- [] All branches merged to develop in dependency order
- [] Full end-to-end scenario passes (Section 21)
- [] Cross-domain PRs (match-accept, handover-confirm) jointly reviewed

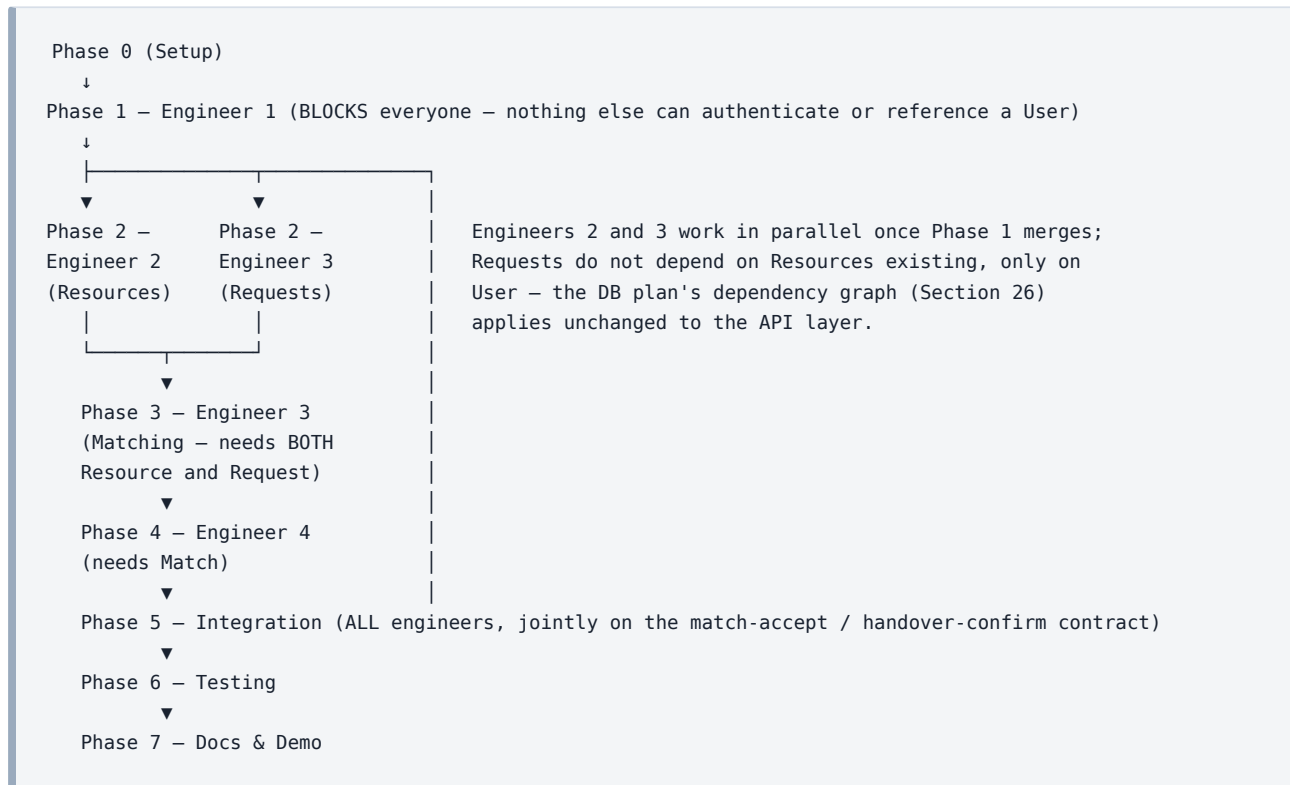
PHASE 6 – Testing & Bug Fixing

- [] Full test suite (Section 18) green
- [] Manual Postman walkthrough of every endpoint in Section 14

PHASE 7 – Documentation & Demo Prep

- [] Postman/OpenAPI docs complete (Section 23)
- [] Seed script produces a demo-ready database (Section 19)
- [] Demo rehearsed against develop (Section 28)

26 — Critical Path



The single longest dependency chain — and therefore the project's critical path — is: **Engineer 1 (auth+User) → Engineer 3 (Request, then Match, which also needs Engineer 2's Resource) → Engineer 4 (Handover) → Integration**. Engineer 2's Resource work is on the critical path only insofar as Matching needs it; Category work is not on the critical path at all and can slip without delaying the team.

27 — Risks & Common Failure Points

Risk	Likely cause	Mitigation
Match acceptance leaves inconsistent state	A write inside the transaction missing <code>{ session }</code>	Explicit code-review checklist item for Task 3.B; the forced-rollback test in Section 18
One-sided handover completion	<code>side</code> derived from client input instead of <code>req.user._id</code>	Named as a specific code-review flag in Task 4.A; a dedicated negative test

Risk	Likely cause	Mitigation
Status fields writable directly via generic <code>PUT</code>	Beginner instinct to accept <code>{ status }</code> in a body update	Lifecycle changes routed exclusively through <code>/status</code> sub-endpoints calling the table-driven services (Sections 08–11)
NoSQL injection via unvalidated query params	<code>Model.find(req.query)</code> shortcuts	Section 15's allow-list rule enforced in code review for every list endpoint
Scope creep toward ML matching or extra infrastructure	Beginner over-engineering instinct	Section 03's explicit "not used" list, reinforced at each phase kickoff
Cross-domain merge conflicts	Two engineers editing the same shared middleware/util file	Section 20's ownership rule for shared files; small, focused PRs
Schema/contract drift from this document	An engineer improvises a new field/status mid-implementation	Section 12/DB-plan-Section-18's "update the document first" rule
Demo-day failure from un-rehearsed flow	Skipping Section 21/28 rehearsal	Mandatory rehearsal against <code>develop</code> before Phase 7 sign-off

28 — Final Graduation Demo Flow

Backend-readiness checklist mirroring the Product Brief's demo narrative (Section 37 of that document), 5–7 minutes, one pre-seeded provider and one pre-seeded seeker/organization account (Section 19):

```

01 Problem    – (frontend narrative; no backend action)
02 Resource   – POST /api/auth/login (provider) → POST /api/resources → PUT ../status {publish}
03 Request    – POST /api/auth/login (seeker/org) → POST /api/requests → PUT ../status {publish}
04 Match      – POST /api/matches/:resourceId/generate → GET /api/matches (ranked, scored)
               → PUT /api/matches/:id/accept (transaction: match+resource+request+handover)
05 Handover   – POST /api/transactions/:matchId/confirm (provider) → still_in_progress
               → POST /api/transactions/:matchId/confirm (seeker) → completed
06 Impact     – GET /api/users/me/contributions (both accounts show updated history)
07 Admin      – POST /api/auth/login (admin) → GET /api/admin/analytics
               (reflects the just-completed transfer, per Product Brief Section 37's "close" beat)

```

Every step above corresponds to a specific, already-built, already-tested endpoint in this plan — the demo requires no code path that was not exercised by the automated test suite (Section 18) beforehand.

This document, together with `Dawwarha – MongoDB Implementation Plan v1.0`, is the team's single Backend Execution Guide from day one of implementation until final graduation-project delivery. If any task appears to require inventing a collection, field, status value, endpoint, or service contract not listed in either document, the correct next step is to update the relevant document first, through the Shared Database Contract (DB plan Section 18) or the Cross-Engineer Contract table (Section 12 above) — not to improvise silently inside one engineer's branch.